Use Cases and User Stories

Project: E-Commerce & Order Management System (Olist) · **Group:** TarushTanishkTanishq
Version: 1.0 (mid-semester) · Traces to the SRS (FR-1 … FR-20)

1. Actors

Actor	PostgreSQL role	Description
Administrator	`olist_admin`	Full access; manages application users
Operations manager	`olist_manager`	Places, updates and cancels orders; manages catalogue data; runs reports and ML
Data analyst	`olist_analyst`	Read-only reporting; cannot see exact customer coordinates
Seller	`olist_seller`	Sees and updates only orders containing its own items (row-level security)
Customer support	`olist_support`	Looks up orders and customers (privacy view); answers reviews
ML training script	`postgres` (offline)	Reads features from SQL views, writes model metrics and predictions

2. Use-case diagram

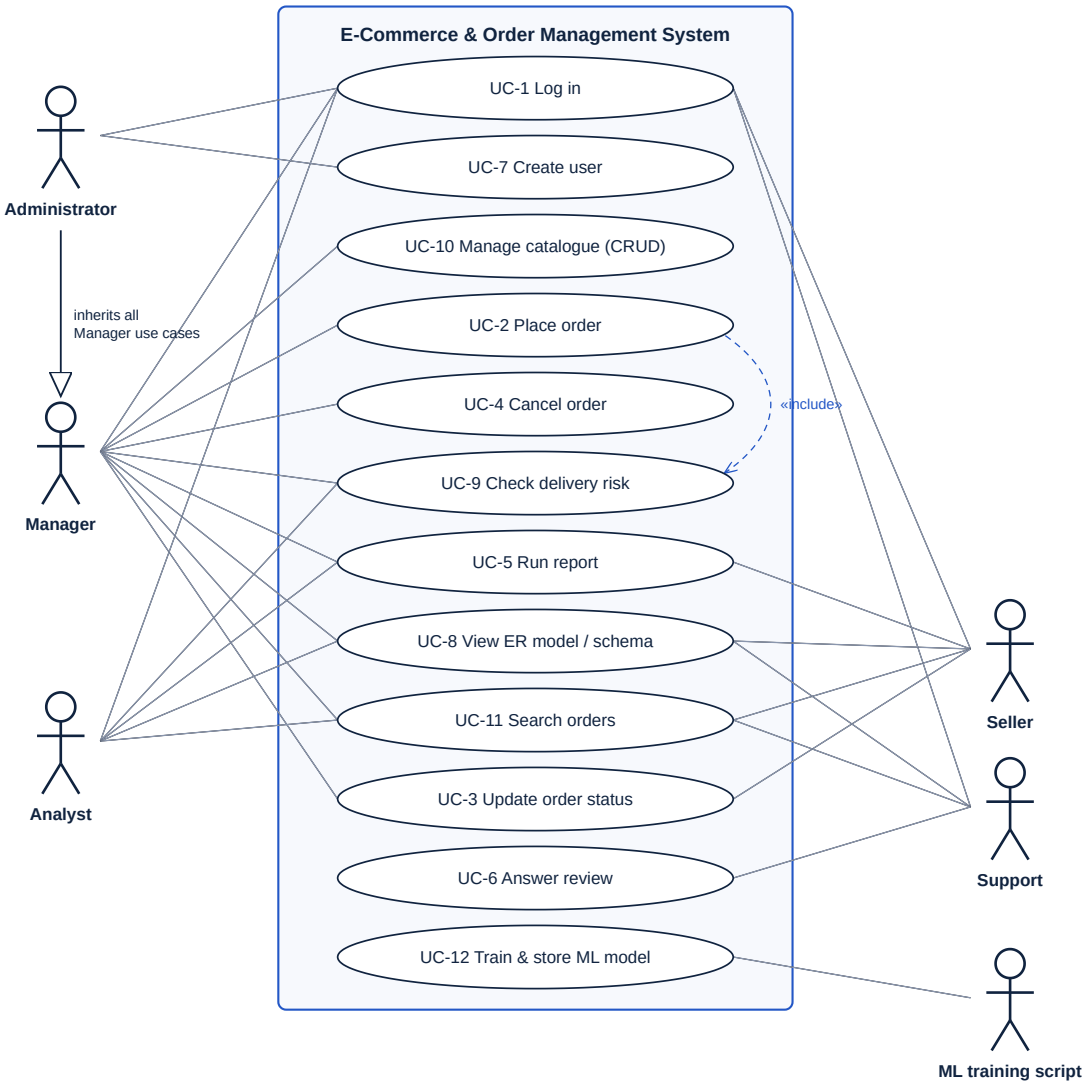


Figure 1: UML use-case diagram. Lines show which roles may perform each use case; PostgreSQL enforces the same permissions.

Every use case except UC-12 requires UC-1 (log in) first. Administrator specialises Manager (generalisation arrow).

3. Use-case descriptions

UC-1 Log in

Field	Description
Actor	Any user
Precondition	User exists in `app_user`; PostgreSQL is running
Main flow	1. User enters username and password. 2. API calls `SELECT * FROM fn_login(u, p)`. 3. PostgreSQL compares the bcrypt hash. 4. API stores user and role in the session. 5. UI shows the menu for that role.
Alternative flow	3a. Wrong password: no row is returned → "Wrong username or password" (HTTP 401).

Field	Description
Postcondition	Every later request runs `SET LOCAL ROLE olist_<role>`
Requirements	FR-1, FR-2

UC-2 Place order

Field	Description
Actor	Manager (or admin)
Precondition	Logged in; customer and products exist; stock > 0
Main flow	1. Manager picks a customer, products and payment type. 2. API runs `CALL sp_place_order(...)`. 3. Procedure inserts the order, its items and payment; trigger reduces stock; trigger writes the audit row. 4. API reads `v_order_features` and scores the order. 5. Prediction is stored in `ml_prediction`. 6. UI shows the order id, risk level and predicted days, plus the SQL executed.
Alternative flows	3a. Unknown customer / product, or stock would go below 0 → error; the whole transaction is rolled back.
Postcondition	Order, items, payment, audit row and prediction exist, or nothing was saved
Requirements	FR-5, FR-7, FR-10, FR-17

UC-3 Update order status

Field	Description
Actor	Seller (own orders only), manager, admin
Main flow	1. User opens an order and chooses the next status. 2. `CALL sp_update_status(id, status)`. 3. Trigger `trg_validate_status_transition` allows only forward moves. 4. Trigger `trg_order_status_log` writes old/new status and role.
Alternative flows	3a. Backward move or change to a final order → error, nothing changed. 1a. Seller opens another seller's order → not visible (RLS).
Requirements	FR-6, FR-7, FR-16

UC-4 Cancel order

Field	Description
Actor	Manager, admin
Main flow	`CALL sp_cancel_order(id)` → status set to `canceled`, audit row written.
Alternative flow	Order already delivered/canceled → error "cannot be canceled".
Requirements	FR-5, FR-7

UC-5 Run report

Field	Description
Actor	Analyst, manager, admin, seller (own data)
Main flow	1. Choose a report Q1–Q16. 2. API runs it in a read-only transaction. 3. Rows and SQL shown. 4. Optional: EXPLAIN ANALYZE plan; CSV export.
Requirements	FR-12, FR-13

UC-6 Answer review

Field	Description
Actor	Support
Main flow	Search reviews → mark answered (`UPDATE review SET answer_ts`); support has a column privilege on `answer_ts` only.
Requirements	FR-4, FR-16

UC-7 Create user

Field	Description
Actor	Administrator
Main flow	Enter username, password, role (and seller id for sellers) → `CALL sp_create_user(...)` stores a bcrypt hash.
Alternative flow	Duplicate username → UNIQUE violation shown.
Requirements	FR-19

UC-8 View ER model / schema

Field	Description
Actor	Any user
Main flow	Click "ER model & schema" → ER diagram, then tabs read live from `information_schema` / `pg_catalog` : tables & keys, constraints, objects, privileges, normalization.
Requirements	FR-14

UC-9 Check delivery risk

Field	Description
Actor	Manager, admin, analyst (read-only)
Main flow	Open the ML page → model metrics, confusion matrix and risk deciles computed in SQL, list of high-risk open orders (`v_high_risk_orders`). "Predict risk only" places a temporary order, scores it and rolls back.
Requirements	FR-17, FR-18

UC-10 Manage catalogue (CRUD) · UC-11 Search orders · UC-12 Train model

UC	Actor	Main flow	Requirements
UC-10	Manager, admin	Insert / search / update / delete products, customers, sellers, reviews, categories; every statement shown in the SQL panel	FR-3, FR-4, FR-13
UC-11	All roles	Filter by status, state, date range, rating, lateness; each dropdown shows the row count for that option	FR-11
UC-12	ML script	Refresh materialized views → time-series CV → pick best model → test on Jun–Aug 2018 → write `ml_model` and `ml_prediction`	FR-17, NFR-10

4. User stories with acceptance criteria

ID	User story	Acceptance criteria	Priority	Status
US-1	As a **manager**, I want to place an order with several items in one step so that the order is never half-saved.	Order, items and payment are inserted together; any error saves nothing (TC-11, TC-12, TC-13).	Must	Done
US-2	As a **seller**, I want to see only my own orders so that other sellers' data stays private.	Seller sees 0 orders without its items; enforced by PostgreSQL RLS, not only the UI (TC-17).	Must	Done
US-3	As a **manager**, I want status changes to be validated and logged so that order history cannot be faked.	Backward moves rejected; every change has an audit row with role (TC-06 – TC-08).	Must	Done
US-4	As an **analyst**, I want predefined reports with their SQL and plan so that I can trust and reuse the numbers.	16 reports run; EXPLAIN and CSV available; CSV rows = UI rows (TC-24, TC-25).	Must	Done
US-5	As an **evaluator**, I want to see every SQL statement a click runs so that I can follow Front end → SQL → DB → Result.	SQL panel lists statement, rows, time and errors for every request (TC-22, all screenshots).	Must	Done
US-6	As a **manager**, I want a late-delivery risk at checkout so that I can act before the customer complains.	Risk level and predicted days shown and stored when an order is placed (TC-30).	Should	Done
US-7	As a **manager**, I want to test "what if" orders without saving them.	Prediction shown, order count unchanged (TC-31).	Could	Done
US-8	As an **analyst**, I must not see exact customer locations.	`zip_code.lat/lng` denied by column privilege (TC-18).	Must	Done
US-9	As an **admin**, I want to create users with a role so that new staff get least-privilege access.	User created with bcrypt hash; can log in with the new role.	Could	Done
US-10	As a **student/evaluator**, I want to rebuild the whole system with three scripts.	`setup_db` → `train_model` → `run_app` works on a clean machine (TC-01, TC-29).	Must	Done
US-11	As a **customer**, I want to order through a storefront.	Out of scope for this release (see SRS scope).	Won't (now)	Not started
US-12	As a **manager**, I want e-mail alerts for high-risk orders.	Candidate for the second half of the semester (currently out of scope).	Could	Backlog